

COMPTE RENDU :Analyse Vidéo Distribuée avec Accélération GPU

1. Contexte et objectifs du projet

Ce projet a pour objectif de mettre en œuvre un système d'analyse vidéo intelligente (application de vidéosurveillance) reposant sur la détection d'objets par le modèle YOLOv8, en simulant une architecture de calcul distribué inspirée du modèle MPI (Message Passing Interface) utilisé en calcul haute performance (HPC).

La vidéo à analyser est découpée en plusieurs portions (« chunks »), chacune étant traitée par un « worker » indépendant, à l'image d'un rank MPI travaillant sur ses propres données locales. Le projet compare ensuite les performances de ce traitement distribué exécuté sur CPU d'une part, et sur GPU (CUDA) d'autre part, afin de quantifier le gain apporté par l'accélération matérielle.

Une dimension pédagogique importante du notebook concerne la résolution de problèmes réels de gestion mémoire : une première version du projet, développée sur Google Colab, provoquait des plantages liés à une consommation excessive de RAM. Le notebook Kaggle documente explicitement les corrections apportées pour fiabiliser l'exécution.

2. Technologies utilisées

Composant	Détail
Langage	Python 3
Détection d'objets	YOLOv8 (bibliothèque ultralytics)
Traitement vidéo	OpenCV (opencv-python-headless)
Calcul distribué (simulation MPI)	multiprocessing.Pool
Accélération matérielle	PyTorch + CUDA (GPU Tesla T4 x2)
Visualisation	Matplotlib (graphiques comparatifs)

Suivi ressources	psutil (usage CPU)
Plateforme d'exécution	Kaggle Notebooks (~30 Go RAM, GPU T4/P100)

3. Architecture du système et étapes MPI simulées

Le traitement suit les quatre grandes étapes d'un pipeline MPI classique :

Étape	Implémentation	Concept MPI équivalent
Découpage	analyze_and_split_video() calcule des indices (start, end) par worker, sans charger aucune frame	MPI_Scatter
Lecture	Chaque worker ouvre lui-même le fichier vidéo et lit uniquement sa portion (streaming)	Accès local aux données (pas de mémoire partagée)
Traitement	process_worker() applique YOLOv8 frame par frame ; multiprocessing.Pool sur CPU, exécution séquentielle sur GPU	Parallélisme inter-rank
Sauvegarde	Chaque frame annotée est écrite immédiatement sur disque (JPEG), sans accumulation en RAM	Sortie locale par rank
Agrégation	assemble_video_from_frames() réassemble les frames triées en une vidéo finale	MPI_Gather + reconstruction

Le notebook précise également comment cette architecture serait transposée sur un vrai cluster HPC, en remplaçant multiprocessing.Pool par la bibliothèque mpi4py et en lançant l'exécution via mpirun, chaque rank MPI correspondant alors à un worker.

4. Problèmes rencontrés et corrections apportées

Le notebook documente explicitement une démarche de débogage et d'optimisation par rapport à une version précédente (développée sur Google Colab) qui plantait systématiquement :

Problème original (version Colab)	Correction appliquée (version Kaggle)
Toutes les frames chargées en RAM (~27 Go)	Chaque worker lit sa portion directement depuis le fichier vidéo (streaming)
4 workers utilisant YOLOv8-Large (~870 Mo chacun)	Réduction à 2 workers pour limiter la charge mémoire
Résolution d'image IMG_SIZE = 1280 (très lourd)	Résolution réduite à IMG_SIZE = 640 (bon compromis qualité/performance)
Frames annotées doublées en mémoire avant assemblage	Sauvegarde directe sur disque (JPEG) au fil du traitement par chaque worker
multiprocessing.Pool utilisé aussi pour le GPU (conflits CUDA)	Exécution séquentielle des workers pour le mode GPU, Pool conservé pour le CPU

À noter : malgré la documentation évoquant l'usage d'un modèle « Nano » (yolov8n.pt, 6 Mo) pour alléger la charge, la configuration réellement exécutée dans le notebook utilise le modèle MODEL_NAME = "yolov8l.pt" (YOLOv8-Large). Ce point mériterait d'être clarifié ou corrigé, le modèle Large étant nettement plus coûteux en mémoire et en temps de calcul que le modèle Nano annoncé dans les commentaires.

5. Déroulement et résultats des tests

Le notebook a été exécuté avec succès sur Kaggle, avec un GPU Tesla T4 x2 disponible (15,64 Go de mémoire, CUDA 12.8). La vidéo de test utilisée faisait 37,7 Mo (597 frames, résolution 2560×1440, 25 FPS, ~23,9 secondes), et le traitement a été limité aux 150 premières frames pour les besoins du test, réparties en 2 workers de 75 frames chacun.

Les résultats mesurés lors de l'exécution sont les suivants :

Métrique	CPU distribué	GPU distribué
Frames traitées	150	150
Temps total de traitement	93,75 s	11,86 s
Vitesse de traitement (FPS)	1,60 img/s	12,65 img/s
Temps moyen par frame	0,594 s	0,048 s

Usage CPU moyen (hôte)	162,9 %	129,6 %
------------------------	---------	---------

Le gain de performance obtenu grâce au GPU est d'un facteur $\times 7,9$ par rapport au traitement CPU, ce qui illustre concrètement l'intérêt de l'accélération matérielle pour des tâches de vision par ordinateur en temps réel, comme la vidéosurveillance.

Trois fichiers de sortie ont été générés avec succès dans `/kaggle/working/` : la vidéo annotée en mode CPU (41,2 Mo), la vidéo annotée en mode GPU (41,2 Mo), ainsi qu'un graphique comparatif regroupant six visualisations (temps total, FPS, distribution du temps par frame pour le CPU et le GPU, temps par worker, et répartition de la charge entre workers).

6. Limites identifiées

- Le parallélisme MPI est simulé via multiprocessing et non une véritable exécution distribuée sur plusieurs machines ; le passage à un vrai cluster nécessiterait une réécriture avec mpi4py.
- Sur GPU, les workers s'exécutent de façon séquentielle (et non en parallèle) pour éviter les conflits CUDA lorsqu'un seul GPU physique est partagé ; l'accélération mesurée provient donc uniquement de la puissance du GPU, pas d'un parallélisme supplémentaire entre workers.
- Incohérence entre la documentation (modèle "Nano" annoncé) et la configuration réellement utilisée (YOLOv8-Large), à corriger pour la cohérence du projet.
- Le test a été limité à 150 frames sur les 597 disponibles ; les résultats extrapolés à la vidéo complète n'ont pas été mesurés.
- Absence de mesure de la précision de détection (mAP, faux positifs/négatifs) : seule la performance temporelle est évaluée, pas la qualité des détections YOLOv8.

7. Conclusion

Ce projet démontre la mise en place d'un pipeline de vision par ordinateur distribué, structuré selon les grandes étapes d'un traitement MPI (scatter, calcul local, gather), appliqué à la détection d'objets en vidéosurveillance avec YOLOv8. Le travail met particulièrement en valeur une démarche d'optimisation mémoire rigoureuse, corrigeant les limitations d'une première version instable, ainsi qu'une comparaison chiffrée et documentée entre exécution CPU et GPU, révélant un gain de performance significatif ($\times 7,9$) grâce à l'accélération matérielle.

Les perspectives d'amélioration incluent une véritable implémentation distribuée avec mpi4py sur un cluster multi-nœuds, l'ajout de métriques de qualité de détection, ainsi que la clarification de la configuration du modèle YOLO utilisé.